武漢大學

本科毕业论文（设计）

嵌入式系统中高效二进制日志管理机制的研究与设计

姓　　名：　　邓泽宇

学　　号：　　2022302111098

专　　业：　　软件工程

学　　院：　　计算机学院

指导教师：　　武小平

二〇二六年三月

原创性声明

本人郑重声明：所呈交的论文（设计），是本人在指导教师的指导下，严格按照学校和学院有关规定完成的。除文中已经标明引用的内容外，本论文（设计）不包含任何其他个人或集体已发表及撰写的研究成果。对本论文（设计）做出贡献的个人和集体，均已在文中以明确方式标明。本人承诺在论文（设计）工作过程中没有伪造数据等行为。若在本论文（设计）中有侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名：_____　　指导教师签名：_____

日　期：_____

版权使用授权书

本人完全了解武汉大学有权保留并向有关部门或机构送交本论文（设计）的复印件和电子版，允许本论文（设计）被查阅和借阅。本人授权武汉大学将本论文的全部或部分内容编入有关数据库进行检索和传播，可以采用影印、缩印或扫描等复制手段保存和汇编本论文（设计）。

作者签名：_____　　指导教师签名：_____

日　期：_____

摘　要

嵌入式与边缘设备中的日志系统既承担故障诊断、运行观测与行为追踪等任务，又受到计算能力、存储容量和并发访问条件的严格约束。现有文本日志方案虽然具备良好的可读性与兼容性，但在高频写入场景下普遍存在格式化开销高、冗余存储多以及共享元数据重复解析的问题；当多个进程或节点共同依赖同一份格式信息时，还容易引入初始化竞争与一致性风险。针对上述矛盾，本文设计并实现了基于共享事件格式的嵌入式二进制日志系统 Optbinlog。

本文围绕“机制设计—理论建模—实验验证”的主线展开研究。首先，针对日志热路径冗长和共享元数据并发初始化不稳定的问题，提出由共享 schema、偏移访问、自描述帧和记录级校验构成的系统方案；其次，建立由日志条数 N 与并发设备数 D 共同驱动的时间、空间与吞吐模型，用于刻画不同模式下固定项、边际项与竞争项之间的关系；最后，在统一统计口径和严格语义对齐条件下，对单机、本地多设备、真实多设备代理场景以及日志量扩展场景进行系统验证。

实验结果表明，Optbinlog 相较文本路径能够稳定降低时间与空间开销；在严格公平口径下，Optbinlog 对 syslog/ulog 表现出稳定的综合优势，而面对 nanolog/zephyr 等极短热路径方案时，单位记录常数项仍有进一步压缩空间；hilog 则在不同层级场景中呈现“空间持续占优、时间与吞吐易受并发路径影响”的过渡特征。空间交叉点实验进一步表明，Optbinlog 在对标 syslog/ulog 时可较早释放空间收益，在对标 hilog 时也具备可观测的反超区间；初始化竞争实验则验证了共享格式文件在多进程与节点级并发场景下的单创建者复用语义。理论推导与实测趋势总体一致，说明本文提出的共享事件格式机制在强调跨进程一致性、结构化表达与长期在线部署的嵌入式场景中具有较好的工程适用性。

关键词：嵌入式日志；二进制编码；共享内存；并发初始化；性能建模

ABSTRACT

Embedded and edge logging systems must provide observability under tight constraints on computation, storage, and concurrent access. Although text-based logging is easy to read and widely compatible, it incurs high formatting overhead, substantial storage redundancy, and repeated metadata parsing in high-frequency workloads. When multiple processes or nodes share the same event description, initialization races and consistency risks further complicate deployment. To address these issues, this thesis designs and implements Optbinlog, an embedded binary logging system built on shared event schemas.

The study follows a mechanism-model-evidence workflow. First, Optbinlog is designed with shared schemas, offset-based access, self-describing frames, and per-record integrity checks so that schema parsing is moved out of the steady-state hot path while cross-process consistency is preserved. Second, analytical models are developed to characterize time, space, and throughput as functions of log count N and concurrent device count D, with explicit separation of fixed costs, marginal costs, and contention terms. Third, the design is evaluated under unified statistical rules and strict semantic alignment across single-node, local multi-device, real multi-node proxy, and log-volume scaling scenarios.

The results show that Optbinlog consistently improves over text-based paths in both time and space. Under strict fairness, it delivers stable overall advantages against syslog/ulog, still leaves room for reducing per-record constant overhead when compared with ultra-short-path peers such as nanolog/zephyr, and shows a mixed profile against hilog where space remains favorable while time/throughput become scenario-sensitive under node-level contention. Space-crossover experiments further indicate that Optbinlog can release storage benefits early when compared with syslog/ulog, and can also surpass hilog beyond a moderate log volume. Initialization-race experiments validate the single-creator reuse semantics of the shared schema file in both multi-process and node-level concurrent settings. Overall, the empirical trends agree well with the theoretical analysis, indicating that the proposed shared-schema mechanism is well suited to embedded deployments that value cross-process consistency, structured semantics, and long-running online operation.

Keywords: embedded logging; binary encoding; shared metadata; concurrency; performance modeling

目录

摘要 I

ABSTRACT II

1 绪论 1

1.1 研究背景与意义 1

1.1.1 嵌入式场景对日志系统的新要求 1

1.1.2 结构化日志与共享元数据的研究意义 1

1.2 国内外研究现状 2

1.2.1 国外研究进展 2

1.2.2 国内研究进展 2

1.2.3 现有研究不足 3

1.3 本文主要工作 3

1.4 论文组织结构 4

2 日志机制与优化空间分析 5

2.1 代表性机制特征 5

2.1.1 追踪与系统日志机制 5

2.1.2 紧凑编码与延迟写出机制 5

2.1.3 嵌入式工程机制 5

2.2 统一分析框架与 peer 对比 6

2.2.1 泛化代价模型 6

2.2.2 代表性 peer 对比 7

2.2.3 优势归纳与可优化点 8

2.3 真实场景需求与适用边界 9

2.4 设计约束与复现边界 10

2.5 本章小结 10

3 系统设计与理论建模 11

3.1 设计目标与总体框架 11

3.2 共享元数据设计与空间模型 12

3.3 热路径设计与时间模型 14

3.4 并发初始化与多设备模型 17
3.5 本章小结 18

4 实验结果与分析 19
4.1 实验环境与评估口径 19
4.1.1 对比对象与平台 19
4.1.2 数据规模与统计口径 19
4.1.3 初始化竞争验证 21
4.1.4 单设备运行面板与观测项 22
4.2 单机场景结果 23
4.2.1 文本基线对照结果 23
4.2.2 peer 对照结果 24
4.3 多设备场景结果 26
4.3.1 本地多设备结果 26
4.3.2 真实多设备结果 29
4.4 理论对照与机制讨论 35
4.4.1 空间交叉点分析 35
4.4.2 机制解释与边界讨论 37
4.5 本章小结 39

5 结论与展望 40
5.1 本文主要工作与结论 40
5.2 未来工作展望 40

参考文献 42

致谢 45

附录 46

1 绪论

1.1 研究背景与意义

1.1.1 嵌入式场景对日志系统的新要求

随着嵌入式设备、边缘网关和轻量控制节点持续承担更复杂的在线业务，日志已经不再只是离线排障时的附属产物，而是系统运行观测、故障定位和行为复核的重要基础设施。[4,12,14,20,28-30] 与通用服务器环境相比，这类设备通常同时受到 CPU 性能、内存容量、持久化空间和实时响应要求的约束，因此日志系统不仅要“能记录”，还要尽可能缩短热路径、控制空间增量，并在长期运行中保持稳定的并发行为。

在此类场景中，日志机制的评价标准也随之发生变化。单纯强调可读性已不足以支撑长期在线部署，系统还必须同时考虑单条记录写入代价、累计存储体积、并发访问一致性以及异常情况下的可恢复性。尤其在多进程协作或多节点代理部署条件下，初始化阶段的共享资源竞争往往会成为影响系统稳定性的关键因素。

1.1.2 结构化日志与共享元数据的研究意义

传统文本日志在工具兼容性和运维生态方面具有明显优势，但在高频写入场景下，字符串格式化、冗余字符存储和重复解析会持续推高时间与空间成本。[1,3,7-8] 围绕降低日志开销，现有研究已经形成了编译期模板提取、无锁缓冲、异步批刷和离线恢复文本等多条技术路径。然而，这些方案通常分别强调极短热路径、高吞吐追踪或后处理能力，对于“共享格式元数据、跨进程一致性与结构化语义可恢复”这一组工程需求，尚缺少统一的机制设计与系统化验证。

基于此，本文将研究重点放在面向嵌入式部署的结构化二进制编码与共享元数据机制上，希望在统一的理论—实证框架下回答三类问题：何种设计能够同时兼顾语义完整性与性能开销，何种并发组织能够保证共享元数据一致性，以及不同场景下 Optbinlog 相对现有方案的优势边界何在。

1.2 国内外研究现状

1.2.1 国外研究进展

国外围绕日志系统优化的研究主要沿三条路线推进。第一类是以 ftrace、LTTng 为代表的追踪链路优化，其核心在于利用 per-CPU lockless ring buffer、tracepoint instrumentation、channel/sub-buffer 等机制降低采集侵入性并提升并发吞吐。[1,3,15] 这类工作更强调“先把事件采上来”，在系统级性能观测和时序分析中具有明显优势。第二类是以 NanoLog 为代表的紧凑编码路径，通过编译期提取静态模板、运行期仅记录动态参数，将调用点代价压缩到更低水平。[8-10] 第三类是以 systemd-journald、Binlog 为代表的结构化存储与恢复路径，强调二进制对象组织、索引、后处理恢复和检索能力，以提升日志管理完整性与运维可用性。[7,16,23]

从总体趋势看，国外研究已经在“采集链路优化、结构化编码、后处理恢复”三个层面形成较完整的技术谱系，但不同路线的优化目标并不相同：追踪类系统重在低侵入采集，紧凑编码类系统重在极短热路径，结构化日志类系统重在管理与恢复能力。对于嵌入式场景中常见的“共享格式文件复用、跨进程一致映射、统一公平评测”问题，仍缺少针对性较强的系统化验证。

1.2.2 国内研究进展

国内相关工作更强调工程可落地性与资源受限条件下的可用性。RT-Thread ulog、OpenHarmony HiLog Lite 等工程方案分别从异步输出、多 backend 适配、轻量消息头和受限字段能力等角度降低嵌入式平台上的日志开销。[12-14] 其中，ulog 更强调“前台快写、后台慢出”的工程组织，HiLog Lite 更强调通过约束字段和简化头部降低资源占用。与此同时，国内学者也从日志分析平台、异常检测、嵌入式日志系统实现等方向开展了较多研究。[24-30] 这类工作为日志应用价值、平台实现与分析方法提供了有益补充。

不过，现有国内研究更多聚焦于日志利用、日志平台实现或具体工程系统，而对“共享元数据机制如何影响写入热路径、空间斜率与并发初始化正确性”这一问题的讨论相对较少。尤其在多进程或多节点条件下，围绕共享格式文件的一致性设计、不同 peer 的适用场景划分以及统一对比方法，仍有进一步研究空间。

1.2.3 现有研究不足

综合国内外研究现状，可以将现有不足概括为以下三点：1. 多数方案围绕单一目标展开优化，难以同时兼顾结构化语义、空间效率和并发一致性。2. 共享元数据复用机制研究不足，对跨进程初始化竞争、偏移一致映射和共享文件生命周期问题关注不够。3. 缺少在统一语义体量、统一统计口径和多层级场景下的直接对比框架，导致不同方案的适用边界与优化空间不易判定。

1.3 本文主要工作

基于上述背景，本文希望解决的并不是单一的“写得更快”问题，而是在语义完整性、并发一致性和综合开销之间寻找可解释、可验证的工程折中。围绕这一目标，本文聚焦以下四个核心问题：

1. 现有代表性日志系统在热路径优化、并发组织与空间控制方面分别采用了哪些机制，其适用边界是什么。
2. Optbinlog 的共享元数据与二进制编码路径能否在保持语义完整性的前提下弥补现有方案缺口。
3. 在日志条数 N 与并发设备（节点）数 D 变化时，时间、空间与吞吐应呈现怎样的理论趋势与交叉关系。
4. 理论预测与实验观测在何处一致、何处偏离，偏离由哪些实现常数项与系统路径因素导致。

围绕上述问题，本文的主要工作与贡献可归纳为以下四点：

1. 设计并实现面向嵌入式多进程场景的 Optbinlog 系统，形成由共享 schema、偏移访问、自描述帧和初始化状态机组成的完整实现方案。
2. 建立由 N 与 D 驱动的统一分析模型，对时间、空间和吞吐的变化趋势、交叉点与竞争放大效应给出可检验表达。
3. 在统一统计规则和严格语义对齐条件下，完成单机、本地多设备、真实多设备代理场景、空间交叉点扫描与初始化竞争验证，形成闭环证据链。
4. 基于实验结果识别出不同 peer 的分组特征与适用边界，说明 Optbinlog 在 syslog/ulog 场景中的稳定优势、在 hilog 场景中的过渡特征，以及在 nanolog/zephyr 场景下仍需继续优化的方向。

1.4 论文组织结构

本文后续结构安排如下：

1. 第 2 章“日志机制与优化空间分析”：围绕代表性机制、peer 对比、场景需求与可优化空间展开分析，为后续系统设计提供依据。
2. 第 3 章“系统设计与理论建模”：将系统实现与理论推导穿插组织，重点说明共享 schema、记录帧、校验与交叉点模型的对应关系。
3. 第 4 章“实验结果与分析”：在单机、本地多设备与真实多设备场景下验证关键结论，并通过日志量扫描给出空间交叉点阈值。
4. 第 5 章“结论与未来展望”：总结主要结论、当前局限与后续优化方向。

2 日志机制与优化空间分析

本章在绪论综述的基础上，进一步围绕代表性 peer 的机制特征、可优化空间和适用边界展开分析。其目的不是重复文献回顾，而是建立“机制特征—场景需求—优化空间”的对应关系，为第 3 章设计与第 4 章实验提供可直接使用的比较框架。围绕这一目的，本文统一从数据结构组织、热路径代价、并发行为和部署边界四个维度展开比较。

2.1 代表性机制特征

2.1.1 追踪与系统日志机制

ftrace 与 LTTng 更强调 tracing 语义，核心优势在于 per-CPU lockless ring buffer、tracepoint instrumentation 和 channel/sub-buffer 等机制。[1,3,15] 其共同思路是把竞争尽量局部化到 CPU 本地或追踪通道内部，以降低采集侵入性并提升并发吞吐。这类机制适合高并发诊断与系统行为追踪，但在应用层结构化日志场景中，往往存在接入复杂、语义映射成本较高的问题。

systemd-journald 则进一步强调结构化对象、索引和检索能力。[7,16] 与 tracing 类系统相比，它更关注“如何组织和管理日志对象”，适合通用系统日志管理，但对于轻量嵌入式设备而言依赖与运行开销相对偏重。

2.1.2 紧凑编码与延迟写出机制

以 NanoLog、Zephyr logging 为代表的方案重点在于压缩热路径常数项。NanoLog 通过编译期提取静态模板、运行期仅打包动态参数，把大量文本恢复代价后移到后台或离线阶段；Zephyr logging 则通过 deferred logging、消息池与后台统一输出，将前台路径缩短为“封包入队”。[4-10]

这类机制在时间和单位记录空间上往往具有优势，尤其适合追求极短热路径或实时任务前台可控性的场景。但从本文关注的问题看，它们对共享元数据一致性、跨进程格式复用以及共享资源首次创建语义关注较少，因此很难直接回答嵌入式多进程部署中的一致性问题。

2.1.3 嵌入式工程机制

RT-Thread ulog 与 OpenHarmony HiLog Lite 更强调嵌入式设备上的工程可部

署性。ulog 通过异步队列和多 backend 机制降低业务线程阻塞，形成“前台快写、后台慢出”的工程路径；HiLog Lite 则通过受限字段与轻量头部降低资源占用。[12-14]

这类方案更贴近设备侧在线调试与系统日志需求，部署门槛较低，也更符合资源受限平台的工程现实。但在语义扩展、共享格式一致性以及统一评测框架方面，仍存在进一步分析空间。

2.2 统一分析框架与 peer 对比

2.2.1 泛化代价模型

为了把不同机制放到同一比较坐标系内，本文将任意日志模式 m 的代价统一写为“固定项 + 边际项 + 并发等待项”的形式。对单设备、单进程场景，累计时间可写为：

$$T_m(N) = A_m + N\,C_m$$

其中 A_m 表示初始化、格式准备、线程唤醒等固定项，C_m 表示单条记录平均时间开销。进一步分解可写为：

$$C_m = c_{\text{lookup},m} + c_{\text{fmt/enc},m} + c_{\text{queue},m} + c_{\text{check},m} + c_{\text{write},m}$$

对应的单机空间模型写为：

$$S_m(N) = S_{0,m} + N\,B_m$$

其中 $S_{0,m}$ 表示元数据、运行时结构和初始化文件等固定空间，B_m 表示单位记录平均字节开销。吞吐写为：

$$Q_m(N) = \frac{1000N}{T_m(N)} = \frac{1000}{C_m + A_m/N}$$

由此可见，当 N 较小时，固定项 A_m 更容易主导总时间；当 N 持续增大时，边际项 C_m 和 B_m 才决定时间、吞吐与空间斜率。对于真实多设备场景，若设备数为 D、每台设备写入 r 条记录，则 wall-clock 时间可进一步写为：

$$E_m(D, r) = A_m^{\text{init}}(D) + r\,C_m + W_m(D, r)$$

$$S_m(D,r) = S_{\text{meta},m}(D) + Dr\,B_m$$

$$Q_m(D,r) = \frac{1000Dr}{E_m(D,r)}$$

其中 $A_m^{\text{init}}(D)$ 表示设备数增长带来的冷启动代价，$W_m(D,r)$ 表示锁竞争、队列拥塞、I/O 饱和和调度噪声等等待项，$S_{\text{meta},m}(D)$ 则表示元数据随设备数增长的方式。若某方案能跨进程共享元数据，则 $S_{\text{meta},m}(D)$ 更接近常数；若每个设备都要独立维护元数据，则该项会更接近线性增长。

这一泛化模型的意义在于：后续讨论 “可优化空间” 时，不再停留在 “某方案更快” 或 “某方案更省空间” 的经验描述，而是明确区分优化究竟作用于 A_m、C_m、B_m 还是 $W_m(D,r)$。

2.2.2 代表性 peer 对比

需要说明的是，前述调研覆盖了更广的国内外系统范围，而第 4 章的定量复现仅选择其中具有代表性的六类 peer：syslog/ftrace/nanolog/zephyr/ulog/hilog。选择依据是机制覆盖度、用户态可比性与实验可复现性。[1,4,8,12-15] LTTng 与 systemd-journald 在本文中承担机制参照作用，不直接进入第 4 章统一口径下的数值对比。

表 2.1 代表性 peer 机制比较

系统	核心机制	典型使用场景	主要优势	主要代价	比较启示
Linux ftrace	内核 tracing ring buffer	内核/驱动诊断、时序追踪	内核路径短、追踪能力强	接入复杂、与业务日志模型存在语义差异	更适合作为 tracing 语义与底层采集路径的参照
Zephyr logging	deferred 入队 + 后台批刷	RTOS 实时任务日志	前台阻塞小	后台线程调度与队列竞争	适合作为 “短热路径 + 后台转移代价” 机制的代表

（续表 2.1）

系统	核心机制	典型使用场景	主要优势	主要代价	比较启示
syslog	文本日志 + 系统日志接口路径	通用 Linux 服务日志	生态成熟、接入通用	文本格式化与系统路径开销较高	适合作为文本展开与系统路径成本的参照
NanoLog	固定字段二进制直写	超高频、可离线恢复文本	调用点开销极低	语义扩展与工具链耦合强	适合作为极小边际时间项的代表
RT-Thread ulog	异步队列 + 多 backend	嵌入式设备在线调试	工程集成灵活	后台文本格式化与后端竞争	适合作为嵌入式异步工程路径的参照
OpenHarmony HiLog Lite	受限字段 + 轻量头	资源受限系统日志	资源占用较低	语义表达能力受限	适合作为轻量头部与受限语义方案的参照

如表2.1所示，各类 peer 并不是围绕同一目标展开优化，而是分别面向 tracing、极短热路径、文本生态兼容或轻量资源占用等不同场景。将它们并置比较的意义，不在于直接判定某一路线“全面更优”，而在于为后续的优化空间分析提供参照坐标。

2.2.3 优势归纳与可优化点

结合表2.1与前述泛化模型，可以将结构化共享场景中的优化方向归纳到若干关键代价项上。

对 syslog/ulog 一组而言，其优势主要体现在接入便利性与工程成熟度，但文本格式化、系统调用路径以及后台文本输出会同时抬高 $c_{\text{fmt/enc},m}$、$c_{\text{write},m}$ 和单位记录空间 B_m。因此，若目标是构建面向长期在线场景的结构化日志机制，首要优化方向应是减少字符串格式化与冗余文本存储，把重复解释成本从热路径移出，并尽量压低冷启动固定项与多设备场景下的等待放大项。

对 nanolog/zephyr 一组而言，其优势更集中在极小的边际时间项 C_m。前者

通过编译期模板提取压缩运行期开销，后者通过 deferred 路径压缩前台写入时间，因此它们在高频写入和大样本条件下更容易保持较高吞吐。由此可见，若希望在结构化语义前提下逼近这组 peer，优化重点应落在边际项本身，即尽可能压缩字段编码常数、记录头部组织开销以及校验与缓冲写出带来的附加成本。

对 hilog/ftrace 一组而言，其优势并不完全位于同一条性能坐标轴上。ftrace 更强调 tracing 语义和底层采集路径，HiLog Lite 则依赖受限语义与轻量头部获得更低资源占用。因此，与这组机制比较时，问题已不再是简单地追求更小的 C_m 或 B_m，而是在保持结构化语义完整性的前提下，合理控制头部大小、字段表达粒度和并发等待项，使方案在应用层业务日志场景中形成更稳定的综合收益。

据此，面向结构化共享场景的候选方案，其可优化空间大致可以归纳为四类：1. 固定项优化：减少初始化、文件打开、格式准备与视图建立等一次性成本。2. 边际时间优化：压缩字段编码、头部构造、校验与缓冲写出的常数开销。3. 单位空间优化：降低元数据冗余、头部字节和字符串字段展开带来的空间斜率。4. 并发等待优化：抑制初始化竞争、缓冲争用与 I/O 饱和对多设备场景的放大效应。

2.3 真实场景需求与适用边界

由上述分析可知，不同真实场景对日志机制的要求并不一致，其主导代价项也随之变化。

若场景为边缘网关、控制器或长期在线代理进程，通常具有日志量大、运行时间长、进程数多以及故障追溯要求强等特点。此时，随着 N 持续增大，C_m 与 B_m 更容易主导整体收益；而在多进程或多节点部署下，$S_{\text{meta},m}(D)$ 与 $W_m(D,r)$ 的差异也会进一步放大。因此，这类场景更适合采用强调共享元数据、一致性控制和结构化恢复能力的日志机制。

若场景为 RTOS 前台任务、启动关键路径或高频实时打点，则关注重点会明显转向极小的 A_m 与 C_m。此时，极短热路径方案往往更具优势；结构化共享型方案若希望进入此类场景，就必须进一步压缩编码、头部与校验常数，并针对轻量运行模式提供更克制的代价配置。

若场景为内核行为追踪、时序诊断或驱动级观测，则 tracing 语义本身往往比应用层结构化字段更重要，ftrace/LTTng 一类机制更适合作为主工具。若场景资源极其受限、字段表达简单且更强调轻量实现，则 HiLog Lite 一类方案也具有现实意义。由此可见，结构化共享型日志机制的适用边界并非“替代所有日志系

统”，而是在需要结构化语义、共享元数据复用、跨进程一致性和真实多设备可验证性的业务日志场景中提供更合理的工程折中。

2.4 设计约束与复现边界

围绕上述比较，本文将设计约束概括为四点：一是保持结构化字段可恢复与记录级校验，不以削弱语义换取表面性能；二是共享元数据初始化必须满足“单创建者 + 多复用者”；三是比较过程必须建立在统一语义体量、统一字段值和统一统计口径之上；四是结论应能在单机到多设备场景中连续解释。

本文的复现边界如下：已完成代表性系统的代码级阅读与机制提取；受平台限制，未对所有外部系统进行全栈原生部署；第 4 章采用 native-like 机制复现与统一统计口径，所有排序结论均在该边界内成立。

2.5 本章小结

本章围绕代表性机制、统一分析框架、peer 比较、场景需求与设计约束展开讨论，建立了后续设计与实验分析所需的理论基础。首先，本文以固定项、边际项、单位空间项和并发等待项为核心，构建了统一的时间、空间与吞吐分析框架，使不同机制能够在同一口径下进行比较。其次，通过对 syslog/ftrace/nanolog/zephyr/ulog/hilog 等代表性 peer 的比较，可以看出，不同系统分别围绕 tracing、极短热路径、文本生态兼容或轻量资源占用等目标展开优化，其优势并不集中在同一代价项上。

在此基础上，本文进一步归纳出结构化共享场景中的四类主要优化方向，即固定项优化、边际时间优化、单位空间优化和并发等待优化；同时指出，不同真实场景的主导因素并不相同，长期在线、多进程协作场景更关注共享元数据与结构化恢复能力，而高频实时打点和 tracing 场景则更强调极短热路径或底层观测能力。最后，结合统一语义体量、统一统计口径和共享初始化一致性等约束，可以得到本章的核心结论：日志机制的优劣具有明显的场景相关性，后续方案设计必须围绕关键代价项展开，并在统一约束下讨论其适用边界。

3 系统设计与理论建模

本章按照“设计要素—流程实现—理论表达”一体化展开，不再将工程实现与公式推导割裂描述。重点围绕 Optbinlog 的共享 schema、记录帧组织与并发初始化机制，给出可直接映射到第 4 章实验指标的模型表达。

3.1 设计目标与总体框架

结合第 2 章的泛化分析框架，Optbinlog 的设计目标可以直接落到四个代价项上：1. 通过共享元数据和视图缓存降低固定项 A_{bin}。2. 通过 typed 编码和紧凑记录帧控制单位空间 B_{bin}。3. 通过 tag cache、顺序字段编码和批量写出压缩边际时间项 C_{bin}。4. 通过初始化状态机和共享文件复用抑制多设备等待项 $W_{\text{bin}}(D, r)$。

系统采用“共享格式文件 + 进程内视图缓存 + 自描述记录帧 + 批量写出”框架。稳定阶段把格式解析、tag 遍历和统计计算迁出热路径，热路径仅保留字段编码、帧构造、校验与缓冲写出。

如图3.1所示，系统整体可以划分为“输入约束—共享核心—输出恢复”三层：左侧输入层给出事件 schema 与部署目标，中间共享核心完成共享元数据管理、视图复用和记录写出，右侧输出层负责日志文件落盘、校验和恢复。图中的四个模块并不是并列堆叠关系，而是从初始化到稳态写入再到读回恢复的递进链路。

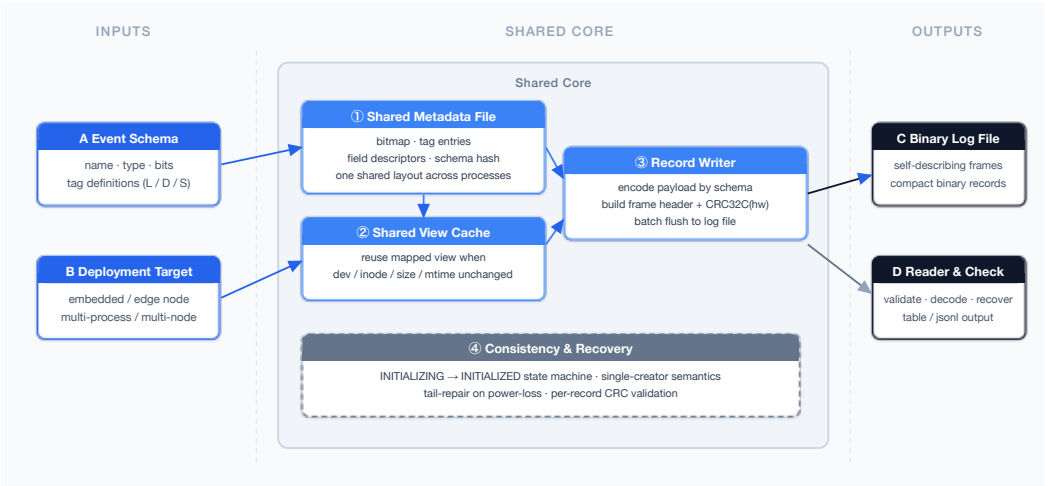

图 **3.1** Optbinlog 系统总体架构

图3.1反映出本文设计的核心思路：把 schema 解释、tag 合法性检查和结构统计尽可能前移到共享核心内部，以换取稳态阶段更短的热路径。换言之，图中的“共享元数据文件与共享视图缓存”主要对应 A_{bin} 与 $S_{\text{meta,bin}}(D)$，“记录写出

器” 主要对应 C_{bin} 与 B_{bin}，“一致性与恢复” 则用于约束 $W_{\text{bin}}(D, r)$ 在并发和异常场景下的放大方式。

3.2 共享元数据设计与空间模型

共享元数据由 OptbinlogSharedTag、OptbinlogBitmap、OptbinlogEventTag、OptbinlogEventTagEle 组成；进程内缓存通过 dev/inode/size/mtime 判定复用，缓存 tag 索引及 schema 统计量。该设计直接决定了单条记录边际空间和共享固定空间两部分。

结合实现代码，共享文件采用连续偏移布局：

$$\text{SharedFile} = [\text{header}][\text{bitmap}][\text{eventtag}][\text{elements}]$$

其中 header 记录 magic/version/state/schema_hash/generation/total_size 等一致性信息，并用 bitmap_offset、eventtag_offset 定位后续段。该布局在初始化阶段先写入 INITIALIZING，全部段落完成并 msync 后再发布为 INITIALIZED，从而保证读写进程只会看到 “完整版本” 或 “等待版本”，避免读取中间态。

记位图段个数为 A、tag 数为 T、字段总数为 E，则共享固定空间可写为：

$$S_{\text{shared}} = \underbrace{|H|}_{\text{header}} + \underbrace{A|B|}_{\text{bitmap}} + \underbrace{T|G|}_{\text{eventtag}} + \underbrace{E|E_l|}_{\text{elements}}$$

其中 $A = \lfloor \max(\text{tag_id})/100 \rfloor + 1$，与实现中 “每组 100 个 tag_id” 的位图分组规则一致。

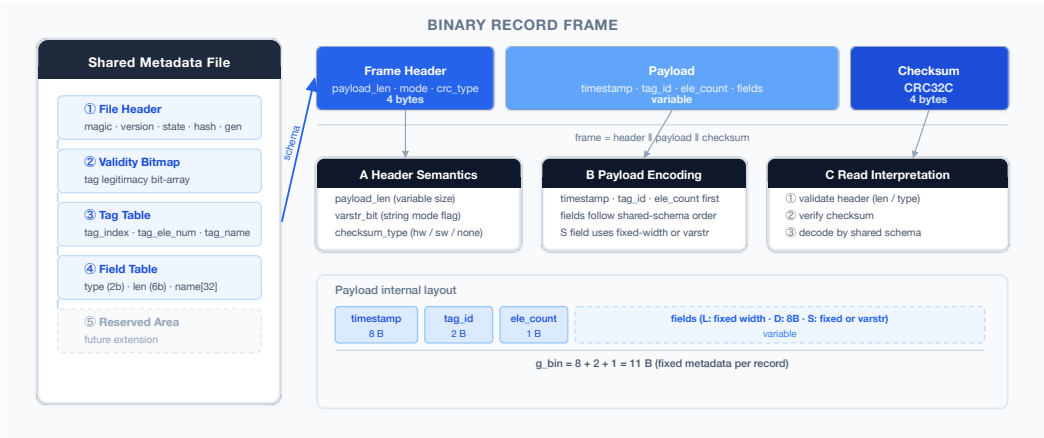

图 **3.2** Optbinlog 共享元数据与记录帧结构

图3.2左侧展示了共享元数据文件的层次组织，包含文件头、合法性位图、tag

表和字段表；这些内容只需在共享文件中保存一次，因此构成总空间中的固定项。右侧则给出了单条二进制记录帧的组织方式，其中帧头负责声明载荷长度和校验模式，payload 负责携带时间戳、tag_id、字段个数和按 schema 顺序编码的字段值，末尾 checksum 用于记录级完整性校验。图中下方给出的 $g_{\text{bin}} = 11\,\text{B}$，正对应后续公式中的每条记录固定元数据开销。

按照第 2 章的统一模型，Optbinlog 的总空间可先写为：

$$S_{\text{bin}}(D, r) = S_{\text{meta,bin}}(D) + Dr\,B_{\text{bin}}$$

其中，Optbinlog 的关键设计点在于把元数据项从 “随设备数重复增长” 改为 “共享文件仅计算一次”，即

$$S_{\text{meta,bin}}(D) = S_{\text{shared}}$$

而对不具备共享元数据机制的方案，往往更接近

$$S_{\text{meta},x}(D) \approx D\,F_x$$

这意味着 Optbinlog 对第 2 章所述 “元数据重复占用” 问题的回应，首先体现在把多设备场景中的固定空间从近线性增长压缩为近常数增长。

当前记录布局为：

$$\text{frame} = \text{frame_header}_{4B} \parallel \text{payload} \parallel \text{checksum}_{4B}$$

其中：

$$\text{payload} = \text{timestamp}_{8B} \| \text{tag_id}_{2B} \| \text{ele_count}_{1B} \| \text{fields}$$

记内部固定元数据 $g_{\text{bin}} = 11$。若字符串字段采用 varstr，其有效载荷写为：

$$\tilde{L}_k = \sum_{j \in \mathcal{N}_k} l_{k,j} + \sum_{j \in \mathcal{S}_k} (2 + \hat{l}_{k,j}), \quad \hat{l}_{k,j} \le l_{k,j}$$

于是单条空间为：

$$b_{\text{bin}}(k) = 4 + g_{\text{bin}} + \tilde{L}_k + 4$$

若将字段类型进一步拆分为长整型字段集合 $\mathcal{L}_k$、双精度字段集合 $\mathcal{D}_k$ 和字符串字段集合 $\mathcal{S}_k$，则可把单位记录开销理解为

$$b_{\text{bin}}(k) = b_{\text{hdr}} + b_{\text{core}} + b_{\mathcal{L}}(k) + b_{\mathcal{D}}(k) + b_{\mathcal{S}}(k) + b_{\text{crc}}$$

其中 $b_{\text{hdr}} = 4$、$b_{\text{core}} = 11$、$b_{\text{crc}} = 4$，而 $b_{\mathcal{S}}(k)$ 是否采用 varstr，直接决定了字符串密集语义组的空间斜率。这正对应第 2 章指出的一个核心优化点：面对 HiLog Lite 或 NanoLog 这类轻量头、紧凑字段方案时，Optbinlog 必须持续压缩 B_{bin}，才能进一步缩小单位记录空间差距。

共享文件只计一次时，总空间为：

$$S_{\text{bin}}(N) = S_{\text{shared}} + N\,B_{\text{bin}}, \qquad B_{\text{bin}} = \sum_k p_k\, b_{\text{bin}}(k)$$

对比模式 x 的空间交叉点可写为：

$$N^*_{\text{space}}(\text{bin}, x) = \frac{S_{\text{shared}} - S_{0,x}}{B_x - B_{\text{bin}}}, \quad (B_x > B_{\text{bin}})$$

若推广到多设备场景，则同样可以写为：

$$S_x(D, r) - S_{\text{bin}}(D, r) = \left[S_{\text{meta},x}(D) - S_{\text{shared}}\right] + Dr\,(B_x - B_{\text{bin}})$$

上述推导与图3.2是一一对应的：左侧共享文件决定 S_{shared}，右侧记录帧决定 $b_{\text{bin}}(k)$ 与 B_{bin}。因此，Optbinlog 的空间优势并不来自“每条记录绝对最小”，而是来自共享元数据只计算一次后，在 syslog/ulog 等高斜率方案面前更容易形成整体空间反超；而在 nanolog/zephyr/hilog 这类低 B_m peer 面前，继续压缩字段与头部仍然是后续设计重点。

3.3 热路径设计与时间模型

写入路径：复用共享视图与 tag cache，按 schema 编码 payload，构造帧头，默认执行 CRC32C(hw)，最后写入用户态批量缓冲区并落盘。读取路径：解析帧头，按声明校验类型校验，依据共享 schema 解码并回调。

为了把流程图与时间模型直接对应起来，图3.3将 Optbinlog 的完整路径拆成三个阶段：阶段 1 负责共享视图建立与复用，阶段 2 负责稳态写入，阶段 3 负责读取、校验与坏尾恢复。其中阶段 1 对应固定项，阶段 2 对应单位记录写入代

价，阶段 3 则说明为什么本文不仅讨论“写得更快”，还要求记录可验证、文件可恢复。

图 **3.3** Optbinlog 写入、读取与恢复流程

从图3.3可以看到，时间代价的主要来源与流程步骤存在明确映射关系：共享文件打开、初始化状态检查和视图复用决定了 t_{lookup} 所在的前置准备代价；按 tag 编码字段值对应 $t_{\text{encode}}(k)$；构造帧头并执行 CRC32C(hw) 对应 $t_{\text{checksum}}(k)$；批量刷入用户态缓冲并调用 write() 则体现在 $\bar{t}_{\text{write}}(b_{\text{bin}}(k))$ 中。图中阶段 3 的读回、校验与坏尾修复步骤虽然不直接计入单次写入性能，却决定了 Optbinlog 在工程上并非“只写不管读”的一次性编码路径。

进一步看查找路径，基础查找函数 optbinlog_lookup_tag 按“位图命中 + 前缀置位计数 + 组内 rank”定位 slot，可写为：

$$\text{slot}(id) = \sum_{i=0}^{arr(id)-1} \text{ones}(bitmap_i) + \text{rank}_{\le idx(id)}(bitmap_{arr(id)}) - 1$$

该路径保证了紧凑存储下的正确映射，但若每条日志都重复执行，会把位图统计代价带入热路径。为此，写入阶段在共享视图建立后一次性构建 tag_id -> {tag, eles, payload_size} 的数组缓存（build_tag_cache），并由 acquire_shared_view 以 (dev,inode,size,mtime) 判定是否可直接复用。于是稳态写入中的 tag 查找退化为数组下标访问，查找开销从“按位图计数”转为近似常数时间：

$$t_{\text{lookup}}^{\text{write}} \approx O(1), \qquad A_{\text{bin}} = a_{\text{open}} + a_{\text{map}} + a_{\text{cache-build/reuse}}$$

这也是第 2 章“固定项压缩 + 边际项压缩”在工程实现上的对应点：把查找计算前移到初始化阶段，将热路径保留为编码与批量写出。

在此基础上，Optbinlog 的固定项可进一步写为：

$$A_{\text{bin}} = a_{\text{open}} + a_{\text{map}} + a_{\text{cache}}$$

其中 a_{open} 对应共享文件打开，a_{map} 对应共享视图建立，a_{cache} 对应 tag 索引与 schema 统计准备。该设计对应第 2 章中“优先压缩 A_{bin}”的优化目标，即把原本可能重复出现在每条日志上的 schema 解释成本，尽量前移并摊薄到固定项中。

对应单条时间可写为：

$$t_{\text{bin}}(k) = t_{\text{lookup}} + t_{\text{encode}}(k) + t_{\text{checksum}}(k) + \bar{t}_{\text{write}}(b_{\text{bin}}(k))$$

进一步地，若将编码阶段按字段类型拆分，则有

$$t_{\text{encode}}(k) = n_{\mathcal{L}}(k)c_{\mathcal{L}} + n_{\mathcal{D}}(k)c_{\mathcal{D}} + n_{\mathcal{S}}(k)c_{\mathcal{S}}(k) + c_{\text{hdr}}$$

于是 Optbinlog 的边际时间项可写为：

$$C_{\text{bin}} = c_{\text{lookup}} + \sum_k p_k\, t_{\text{encode}}(k) + c_{\text{crc}} + c_{\text{bufwrite}}$$

总时间采用“固定项 + 边际项”形式：

$$T_m(N) = A_m + N\,C_m, \qquad C_m = \sum_k p_k\, t_m(k)$$

吞吐表达为：

$$Q_m(N) = \frac{1000N}{T_m(N)} = \frac{1000}{C_m + A_m/N}$$

当比较 bin 与模式 x，且 $C_x > C_{\text{bin}}$ 时，时间交叉点为：

$$N^*_{\text{time}}(\text{bin}, x) = \frac{A_{\text{bin}} - A_x}{C_x - C_{\text{bin}}}$$

该表达解释了“小样本阶段固定项主导、中大样本阶段边际项主导”的实测现象，也说明图3.3中的阶段 1 与阶段 2 在不同日志量区间会轮流成为主导因素。

更重要的是，这一分解可以直接对应第 2 章总结的 peer 优势。对 syslog/ulog

而言，Optbinlog 的优势主要来自把文本格式化代价替换为类型化编码，即在大多数语义组上满足 $C_x > C_{\text{bin}}$；对 nanolog/zephyr 而言，当前差距则集中在 c_{hdr}、c_{crc} 和字符串编码常数上，这也正是第 2 章提出的主要优化方向。换言之，第 2 章中的“可优化点”并不是抽象判断，而是分别对应 A_{bin} 和 C_{bin} 内部仍可继续压缩的具体项。

3.4 并发初始化与多设备模型

为保证共享资源首次创建正确性，Optbinlog 使用 INITIALIZING → INITIALIZED 状态机，约束“单创建者 + 多复用者”语义。与此同时，系统提供 optbinlog_read 做记录级复核，并在掉电场景下执行“坏尾检测-截断-续写”闭环恢复。

按照第 2 章的统一模型，多设备下总时间可先写为：

$$E_m(D,r) = A_m^{\text{init}}(D) + r\,C_m + W_m(D,r)$$

对 Optbinlog 而言，初始化状态机使冷启动项不再表现为“每个设备都完整创建一遍共享文件”，而更接近：

$$A_{\text{bin}}^{\text{init}}(D) = D\,a_{\text{open}} + a_{\text{create}} + (D-1)a_{\text{attach}}$$

其中 a_{create} 仅由单创建者承担，其他设备只执行附着与复用代价 a_{attach}。因此，相较于“每设备重复创建”的路径，Optbinlog 在初始化阶段大致可以减少

$$\Delta A^{\text{init}}(D) \approx (D-1)\big(a_{\text{create}} - a_{\text{attach}}\big)$$

这正是第 2 章中“压制 $W_{\text{bin}}(D,r)$ 和冷启动固定项”的具体设计体现。

将其带入总时间，可得：

$$E_{\text{bin}}(D,r) = D\,a_{\text{open}} + a_{\text{create}} + (D-1)a_{\text{attach}} + r\,C_{\text{bin}} + W_{\text{bin}}(D,r)$$

其中 $W_{\text{bin}}(D,r)$ 表示初始化等待、锁竞争、缓冲争用与 I/O 饱和等等待项。工程拟合可近似为：

$$E_m(D,r) \approx \beta_m + rC_m + \alpha_m D + \gamma_m D^2$$

对应的集群吞吐可写为：

$$Q_m(D,r) = \frac{1000Dr}{E_m(D,r)}$$

吞吐峰值设备数近似：

$$D_{\text{peak},m} = \sqrt{\frac{\beta_m + rC_m}{\gamma_m}}$$

空间模型则可直接写为：

$$S_{\text{bin}}(D,r) = S_{\text{shared}} + Dr\,B_{\text{bin}}$$

它表明，Optbinlog 通过共享元数据把多设备场景中的固定空间压成一次性代价，而真正随设备数放大的只有记录本身。于是，第 2 章中针对真实使用场景提出的两个关键设计点就都在这里体现出来：一是通过状态机把初始化竞争收敛到“单创建者 + 多附着者”，二是通过共享 schema 把多设备元数据重复占用压缩为近常数项。

上述模型与初始化竞争实验、单机/多设备扫描和空间交叉点扫描构成统一映射，为第 4 章结果解释提供直接依据。

3.5 本章小结

本章围绕共享元数据、记录帧组织、热路径写读流程以及并发初始化机制，完成了 Optbinlog 的系统设计与理论建模。首先，通过共享 schema、视图缓存和自描述记录帧，明确了系统在空间维度上的基本结构，并给出了由共享元数据项与单位记录空间项共同构成的空间模型；其次，通过对字段编码、帧头构造、校验与缓冲写出的拆分，建立了由固定项 A_{bin}、边际项 C_{bin} 和吞吐表达共同构成的时间模型；再次，通过 INITIALIZING → INITIALIZED 状态机与共享文件附着机制，进一步给出了多设备条件下冷启动项、等待项和集群吞吐的表达方式。

综合本章分析可以看出，Optbinlog 的设计目标与理论表达是对应的：共享元数据机制主要作用于固定项与元数据空间项，紧凑记录帧和类型化编码主要作用于边际时间项与单位空间斜率，而初始化状态机则用于约束并发场景下的等待放大效应。由此，时间、空间与多设备行为被纳入同一解释框架之中，为后续实验结果的分析提供了清晰的理论依据。

4 实验结果与分析

本章依据第 3 章建立的时间、空间与并发模型，按照“实验环境与评估口径—单设备过程观测补充—单机场景性能分析—多设备场景性能分析—综合结果讨论与理论对照”的顺序展开。主证据采用最新一版严格语义对齐数据，统一比较对象为 text_semantic_like / binary / peer native mode，以保证不同模式之间具有直接可比性。

4.1 实验环境与评估口径

4.1.1 对比对象与平台

本文在实验层面统一比较三类对象：文本基线 text_semantic_like、本文实现的 binary 路径，以及保持原生存储方式的代表性 peer 模式。场景覆盖单机高负载、本地多设备模拟以及节点级真实多设备代理三种层次。[20] 其中，single/multi 场景的 peer 包括 syslog、nanolog、zephyr、ulog、hilog；节点级真实多设备场景进一步纳入 ftrace 作为 tracing 类参照。

运行平台分为两类：单机与本地多设备实验运行于 macOS 26.3（Darwin 25.3.0，ARM64）；真实多设备实验运行于 Lima Ubuntu 24.04 ARM64 虚拟机集群，最多扩展到 20 个节点，每节点配置 2 vCPU。为避免把实现路径差异误判为结果差异，本文统一要求 binary 默认执行 CRC32C(hw)，并在节点侧按 -march=armv8-a+crc 编译；与此同时，真实多设备场景采用“全节点共享同一 shared_eventtag.bin 格式缓存文件、节点结果文件独立输出”的方式，以观察共享元数据在并发访问下的实际收益。

4.1.2 数据规模与统计口径

为保证评估口径一致，本文在所有主实验中统一使用同一字段值生成器、同一语义组和统一的统计流程。主统计指标包括 mean、median、p95、std 与 95%CI；在输出统计结果之前，先按 IQR 1.5 倍规则过滤异常值，再计算时间、空间、吞吐以及相对改进百分比。空间交叉点实验额外采用 $N = 1 \sim 100000$ 的记录数扫描，用于回答不同语义组下何时发生空间反超。

表 4.1 本章统一实验设计与评估口径

维度	本文实验口径
对比对象	text_semantic_like（文本基线）、binary（Optbinlog 最终路径）；single/multi peer：syslog、nanolog、zephyr、ulog、hilog；l1 peer：syslog、ftrace、nanolog、zephyr、ulog、hilog；peer 侧保持原生存储路径
单机/本地多设备平台	macOS 26.3（Darwin 25.3.0，ARM64）
真实多设备平台	Lima Ubuntu 24.04 ARM64 虚拟机集群；最多 20 节点；节点 CPU 为 2 vCPU（aarch64）
校验实现与硬件路径	binary 默认每条记录 CRC32C(hw)；L1 节点 aarch64 CPU flags 包含 crc32，基准按 -march=armv8-a+crc 编译
真实多设备数据量	每节点 records=20000；节点规模 5/10/15/20；repeats=3，warmup=1
真实多设备总样本	六个语义组 × 三种模式 × 节点扫描后，有效统计样本约 54,000,000 条日志（不含 warmup）
真实多设备总日志字节	有效轮次聚合约 4.601 GiB（按 cluster_total_bytes 汇总）
真实多设备空间口径	集群总空间；共享格式文档在每个模式下仅计算一次
空间交叉点附加扫描	records 从 1 到 100000 共 13 个点；repeats=2；指标为 total_bytes

后续各节均以表4.1为统一口径；严格公平层用于主结论，工程近似层仅用于机制补充解释。

4.1.3 初始化竞争验证

在进入性能对比之前，本文首先验证共享格式文件并发初始化的正确性，以确保后续 single/multi/l1 指标建立在一致的并发语义之上。[1,4,15]

1. 本地多进程初始化竞争。

   平均耗时为 12.337 ms，$p95$ 为 13.679 ms，平均创建成功进程数为 1.0，平均复用已存在共享文件成功进程数为 9.0。

2. 节点级真实多设备代理场景下的初始化竞争。

   全局平均耗时为 6480.706 ms，$p95$ 为 6495.396 ms，平均单创建者数量为 1.0，平均失败节点数为 0.0。

两组结果均稳定满足“单创建者、其余进程或节点复用已初始化共享文件”的正确性约束，说明共享元数据机制在本地与节点级并发下均保持一致行为。基于该前置验证，后续性能实验按“单机基线 → 本地多设备 → 真实多设备 → 日志量扩展”的顺序展开，以避免将初始化异常误判为性能差异。

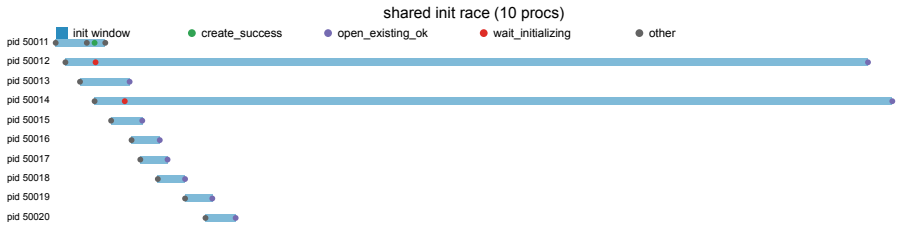

图 **4.1** 本地多进程初始化竞争时序图

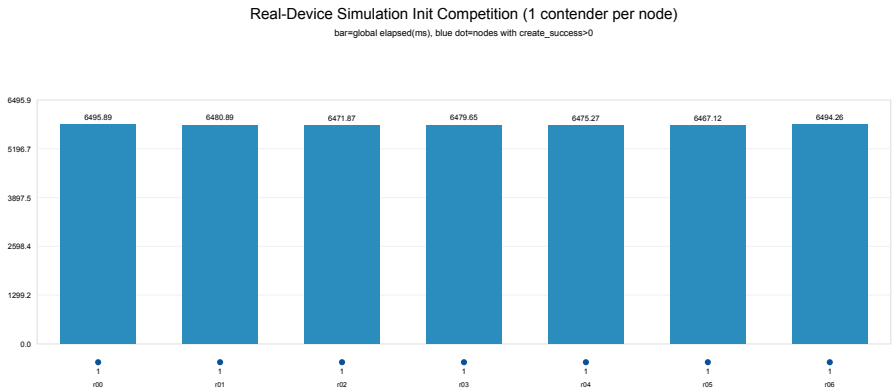

图 **4.2** 真实多设备节点级初始化竞争总览图

4.1.4 单设备运行面板与观测项

除统一统计实验外，本文补充了一个面向单设备本地运行的过程观测面板，用于把“写入路径—异常处理—恢复结果”放在同一时间轴上观察。该部分的目标是增强机制可解释性，不用于替代表4.4至表4.7中的排序结论。

该面板对应一台本地嵌入式运行模拟器，核心观测对象是共享格式文件与运行日志文件的实时演化。运行过程中，控制侧可执行暂停、继续、单步、异常注入与断电截断/修复；观测侧同步展示阶段状态、事件时间线、二进制帧与可读日志联动、共享格式结构解析与关键优化指标。其设计目的，是把第 3 章中的关键变量在运行过程里对应到可直接查看的证据项。

表 4.2 单设备运行面板的观测项与验证问题映射

面板模块	主要观测项	对应验证问题
运行控制	暂停/继续/单步、注入 fault/diag/recover、断电截断与修复	异常与恢复链路是否可控、可重放
时间线与状态	启动/网络/运行/异常/恢复阶段与标注	状态迁移是否与事件写入顺序一致
日志联动	二进制帧映射、解码 tail、异常高亮	run.bin 帧级变化能否被稳定解码与定位
格式结构与指标	shared.bin 布局、schema_hash、空间比、解码耗时	共享元数据与空间收益是否可被在线观测

图4.3展示单设备本地运行时的面板整体状态，用于说明状态迁移、异常标注与帧级解码结果可以在同一界面内对应观察。

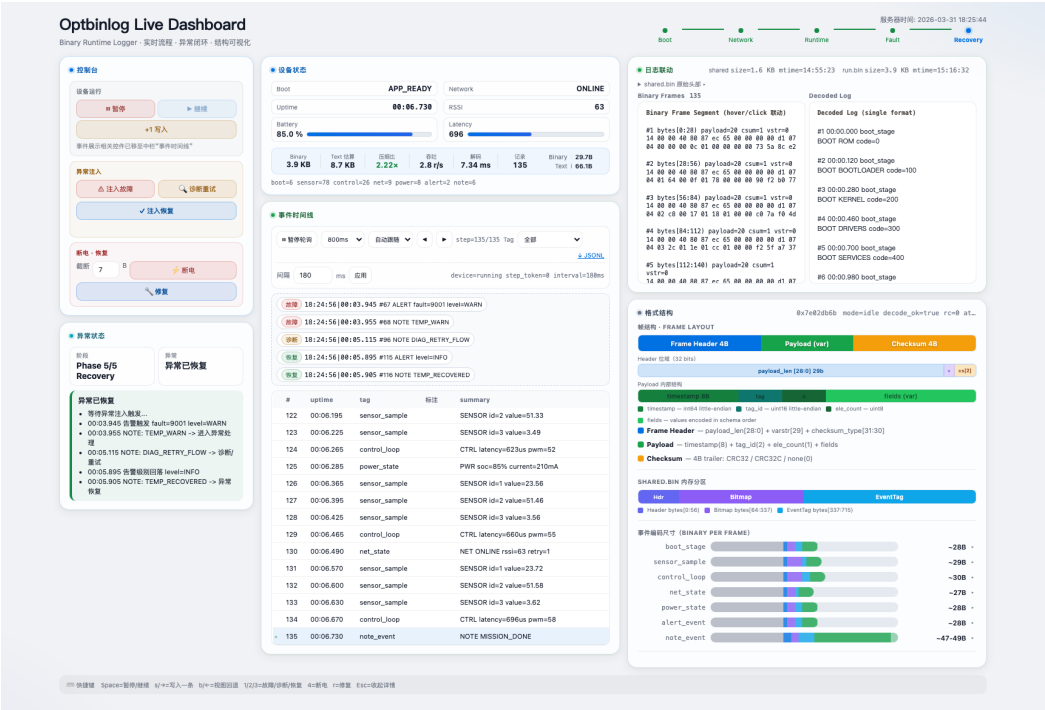

图 **4.3** 单设备本地运行面板整体视图

在输出证据上，该面板会持续写出状态快照、解码结果与运行输出，可用于复核“事件写入—状态变化—恢复执行”的一致性。需要说明的是，这一单设备观测链路主要服务于机制验证与过程说明，不承担多设备扩展性结论；多设备结论仍以 4.3 节统一口径结果为准。

4.2 单机场景结果

为尽量剥离并发等待项的影响，本节首先考察单机高负载条件下 binary 相对文本基线与代表性 peer 的表现。该部分结果对应第 3 章的单机模型，重点观察固定项 A_m、边际项 C_m 与单位空间斜率 B_m 的综合作用。

4.2.1 文本基线对照结果

表 **4.3** 单机高负载下 binary 相对 text 的结果

分组	binary 相对 text 时间%	binary 相对 text 空间%	binary 相对 text 吞吐%	结论
syslog	+73.29%	+45.75%	+274.20%	相较 text 呈稳定显著正收益

（续表 4.3）

分组	binary 相对 text 时间%	binary 相对 text 空间%	binary 相对 text 吞吐%	结论
nanolog	+93.01%	+69.79%	+1335.62%	相较 text 呈稳定显著正收益
zephyr	+93.50%	+69.52%	+1438.78%	相较 text 呈稳定显著正收益
ulog	+93.72%	+68.96%	+1487.27%	相较 text 呈稳定显著正收益
hilog	+93.78%	+71.95%	+1508.37%	相较 text 呈稳定显著正收益

如表4.3所示，binary 相对文本基线在全部语义组上均表现为时间缩短、空间降低和吞吐提升，说明结构化二进制编码替代文本格式化后，能够在单机高频写入条件下形成稳定的基线优势。这一结果与第 2 章对 syslog/ulog 类文本路径的分析一致，即文本展开带来的边际时间和单位空间成本会在高负载条件下快速累积。

4.2.2 peer 对照结果

表 **4.4** 单机高负载下 binary 相对 peer 的结果

分组	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	结论
syslog	+74.17%	+45.75%	+287.00%	三指标同向为正，优势稳定
nanolog	+21.21%	-41.76%	+27.37%	时间与吞吐转正，空间仍为负区

（续表 4.4）

分组	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	结论
zephyr	+25.12%	-41.76%	+33.34%	时间与吞吐转正，空间劣势持续
ulog	+83.73%	+59.43%	+515.11%	三指标同向为正，综合优势显著
hilog	+77.29%	+32.61%	+340.10%	三指标同向为正，综合优势明确

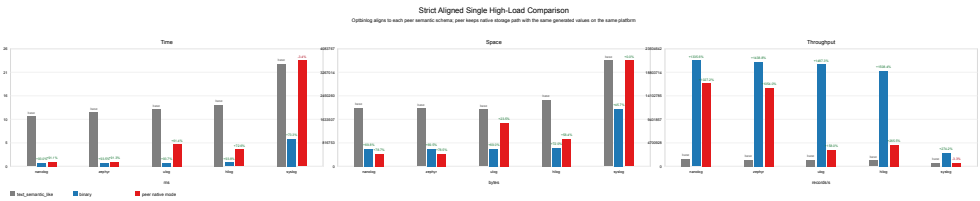

图 **4.4** 严格语义对齐单机高负载总览

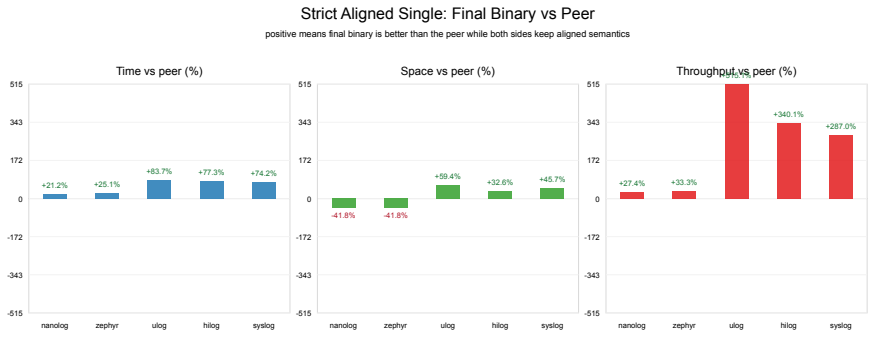

图 **4.5** 严格语义对齐单机场景中 binary 相对 peer 的差值

结合表4.4与图4.4、图4.5可以看到：相对 syslog/ulog/hilog，binary 在单机阶段已形成较稳定的综合正收益；相对 nanolog/zephyr，则表现为时间与吞吐转正，

但空间仍处于负区。由此可见，单机阶段的主要优势来自格式化开销的压缩，而在紧凑二进制路径面前，单位记录常数项仍是主要约束。

4.3 多设备场景结果

在第 3 章模型中，设备数增加主要改变等待项 $W_m(D,r)$ 以及多设备场景下的元数据增长方式。本节分别给出本地多设备与节点级真实多设备结果，以考察结论在更复杂部署条件下是否仍然成立。

4.3.1 本地多设备结果

本节结果同样来自 final_aligned_summary.json，但切换到 multi 部分，并对 $D = 5, 10, 20, 50$ 进行扫描。该场景保持平台不变，只让设备数增加，因此能够单独观察 $W_m(D,r)$ 对趋势的影响。

表 4.5 本地多设备模拟下 binary 相对 peer 的扫描结果

分组	设备数 D	binary 相对 peer 时间%	binary 相对 peer 吞吐%	binary 相对 peer 空间%	现象
syslog	5	-43.04%	-11.31%	+45.68%	低并发阶段时间与吞吐为负
syslog	10	+9.31%	+10.28%	+45.72%	中并发转入稳定正收益
syslog	20	+7.35%	+7.92%	+45.73%	中高并发保持正区
syslog	50	+9.79%	+10.79%	+45.74%	高并发仍处正区
nanolog	5	+2.94%	+1.88%	-42.12%	时间吞吐小幅为正，空间负区

（续表 4.5）

分组	设备数 D	binary 相对 peer 时间%	binary 相对 peer 吞吐%	binary 相对 peer 空间%	现象
nanolog	10	-1.48%	-1.41%	-41.90%	中并发回落至零点附近
nanolog	20	+0.75%	+0.76%	-41.78%	小幅回正但空间未变
nanolog	50	+14.47%	+11.33%	-41.71%	时间吞吐转强，空间仍负
zephyr	5	-5.51%	-4.93%	-42.14%	低并发轻度负收益
zephyr	10	+1.00%	+1.01%	-41.90%	中并发接近持平并微正
zephyr	20	-7.87%	-5.89%	-41.78%	竞争放大后再次转负
zephyr	50	+13.53%	+10.95%	-41.71%	高并发回升为正
ulog	5	+1.19%	+1.17%	+59.32%	低并发即小幅转正
ulog	10	+11.74%	+12.15%	+59.39%	中并发优势扩大

（续表 4.5）

分组	设备数 D	binary 相对 peer 时间%	binary 相对 peer 吞吐%	binary 相对 peer 空间%	现象
ulog	20	+6.74%	+7.05%	+59.42%	中高并发保持正区
ulog	50	-10.21%	-6.32%	+59.44%	高并发时间吞吐回落
hilog	5	+1.92%	+2.03%	+32.41%	低并发小幅领先
hilog	10	-1.88%	-1.83%	+32.53%	中并发短暂转负
hilog	20	+0.67%	+0.66%	+32.59%	中高并发近持平
hilog	50	+1.67%	+1.49%	+32.63%	高并发小幅回正

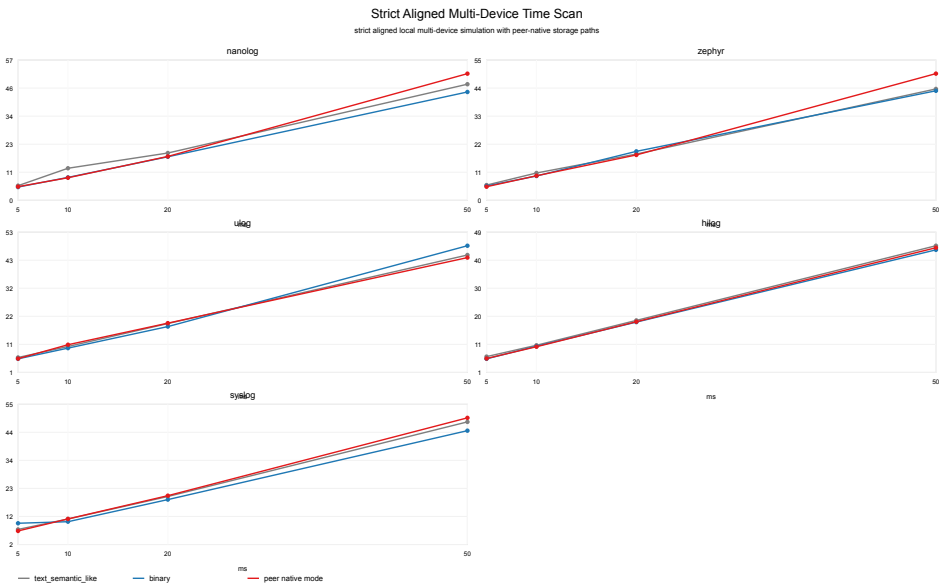

图 **4.6** 严格语义对齐本地多设备时间扫描

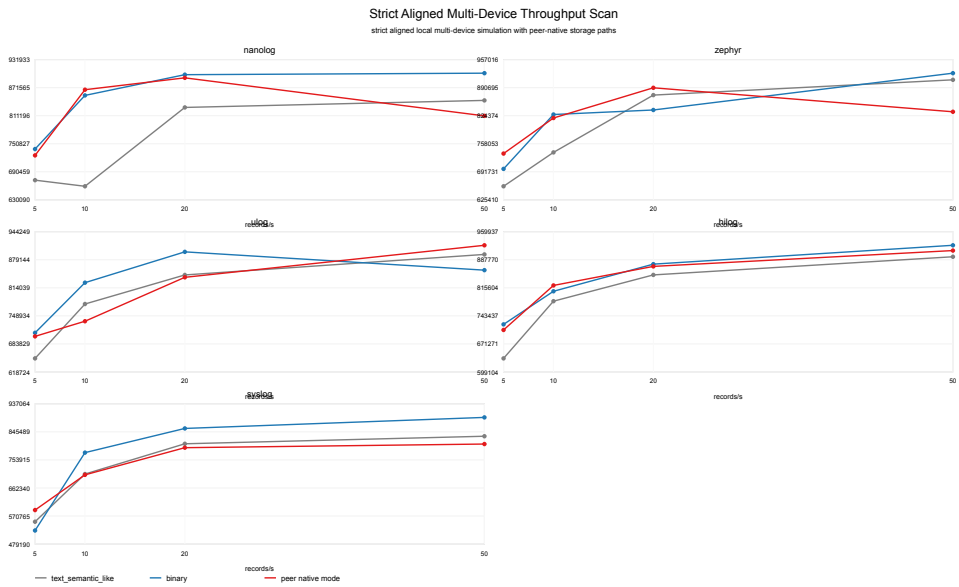

图 **4.7** 严格语义对齐本地多设备吞吐扫描

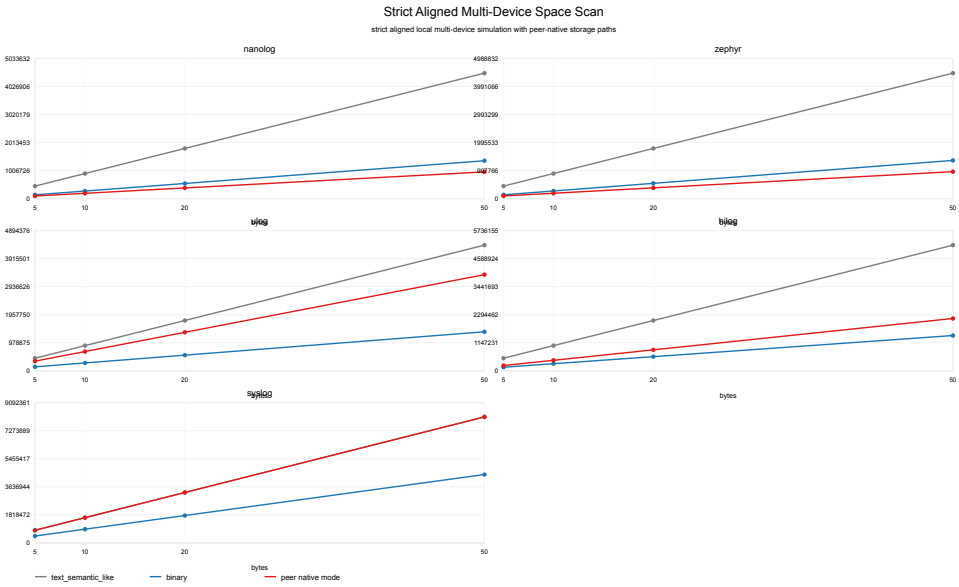

图 **4.8** 严格语义对齐本地多设备空间扫描

如表4.5以及图4.6、图4.7、图4.8所示，本地多设备阶段呈现出“时间与吞吐波动、空间差值稳定”的总体特征。这说明设备数增长首先放大的是竞争等待项，而非单位空间斜率。因此，本地多设备结果主要用于揭示 $W_m(D,r)$ 的变化趋势，而不能直接替代真实多设备部署下的最终结论。

4.3.2 真实多设备结果

本节结果来自严格对齐套件的 11 节点结果及其合并汇总，采用 5/10/15/20 个并发节点、每节点一台设备的扫描方式。与本地多设备不同，这一场景在保

持语义完全对齐的同时，引入了节点级冷启动、VM 调度噪声和跨节点执行偏移，因此更接近论文中的“工程部署代理场景”。该组真实多设备复测统一使用硬件 CRC32C（aarch64 + -march=armv8-a+crc），并额外约束“全节点共享同一 shared_eventtag.bin 格式缓存文件”，用于验证共享元数据在节点级并发下的趋势变化。为便于与前文表格保持一致，表4.6先给出 20 节点最大规模的 binary vs text 截面，表4.7再给出 binary vs peer 的节点扫描结果。

表 4.6 真实多设备模拟下 binary 相对 text 的结果

分组	节点有效数	binary 相对 text 时间%	binary 相对 text 空间%	binary 相对 text 吞吐%
syslog	20/20	+61.26%	+45.76%	+140.07%
ftrace	20/20	+32.42%	+56.12%	+44.40%
nanolog	20/20	+33.61%	+69.81%	+51.02%
zephyr	20/20	+35.69%	+69.54%	+52.16%
ulog	20/20	+33.40%	+68.98%	+49.73%
hilog	20/20	+45.00%	+71.97%	+77.14%

表 4.7 真实多设备模拟下 binary 相对 peer 的结果

分组	节点数	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	现象
syslog	5	+62.49%	+45.76%	+129.90%	三指标同向为正
syslog	10	+58.26%	+45.76%	+122.45%	中节点规模稳定领先
syslog	15	+55.30%	+45.76%	+119.28%	节点提升后优势延续
syslog	20	+63.04%	+45.76%	+128.09%	20 节点仍为稳定正区

（续表 4.7）

分组	节点数	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	现象
ftrace	5	-201.92%	+65.81%	-66.25%	空间为正，时间吞吐为负
ftrace	10	-217.27%	+65.81%	-67.80%	“空间优、时间劣”结构稳定
ftrace	15	-157.97%	+65.81%	-60.76%	结构方向保持一致
ftrace	20	-185.93%	+65.81%	-65.73%	20 节点结构未改变
nanolog	5	-203.24%	-41.68%	-67.09%	紧凑路径优势显著
nanolog	10	-176.39%	-41.68%	-65.03%	负区保持但幅度有波动
nanolog	15	-196.70%	-41.67%	-67.38%	节点提升后再次回落
nanolog	20	-196.45%	-41.67%	-66.28%	20 节点仍未接近交叉
zephyr	5	-100.24%	-41.69%	-50.25%	deferred 路径保持占优

（续表 4.7）

分组	节点数	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	现象
zephyr	10	-104.54%	-41.68%	-52.07%	阶段性波动但未转正
zephyr	15	-134.20%	-41.67%	-55.74%	节点增加后进一步回落
zephyr	20	-115.32%	-41.67%	-53.42%	20 节点仍为负区
ulog	5	+15.79%	+59.45%	+16.10%	低节点已整体转正
ulog	10	+12.10%	+59.45%	+17.60%	中节点优势稳定
ulog	15	+15.81%	+59.45%	+19.98%	节点增加后仍稳定为正
ulog	20	+14.47%	+59.45%	+15.31%	20 节点仍具综合优势
hilog	5	-22.95%	+32.65%	-18.65%	时间吞吐为负，空间为正
hilog	10	-11.56%	+32.66%	-12.58%	负区收敛但未转正

（续表 4.7）

分组	节点数	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	现象
hilog	15	-18.73%	+32.66%	-14.04%	节点提升后再次回落
hilog	20	-13.02%	+32.66%	-12.36%	20 节点仍呈“空间优、时间吞吐负”

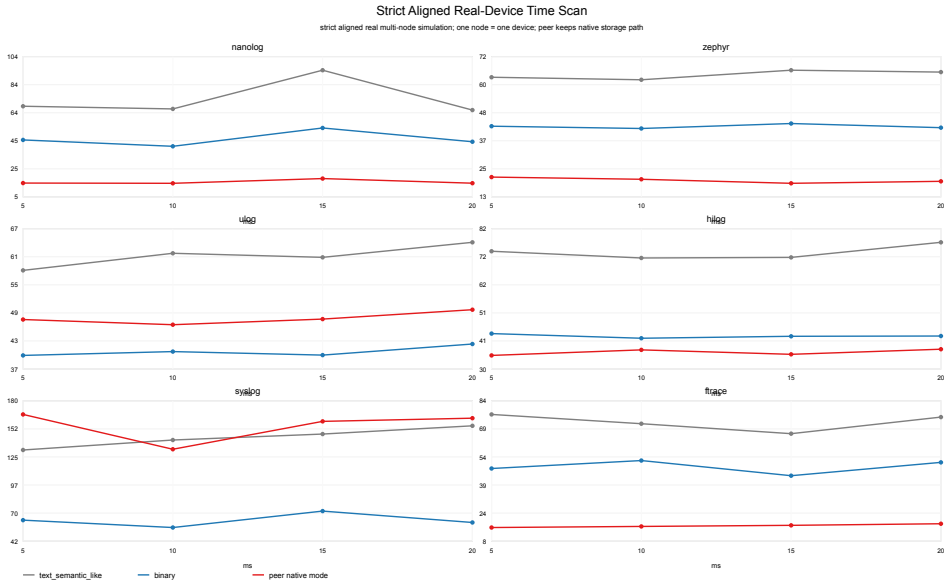

图 **4.9** 严格语义对齐真实多设备时间扫描

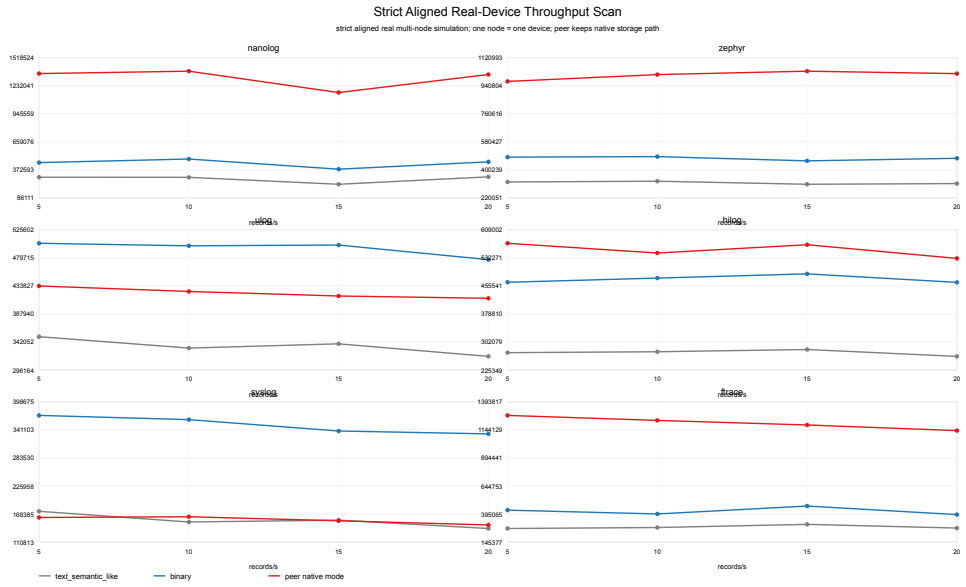

图 **4.10** 严格语义对齐真实多设备吞吐扫描

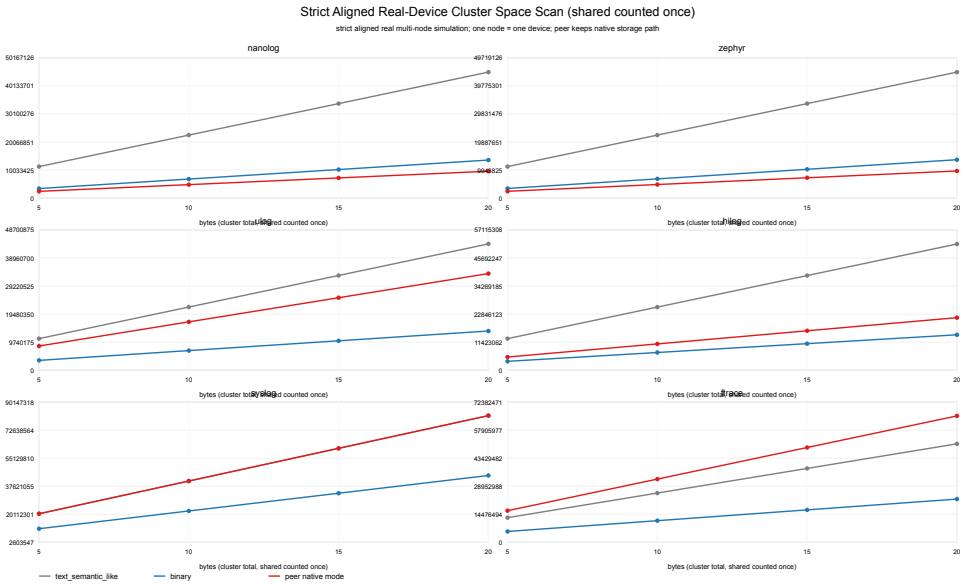

图 **4.11** 严格语义对齐真实多设备集群总空间扫描

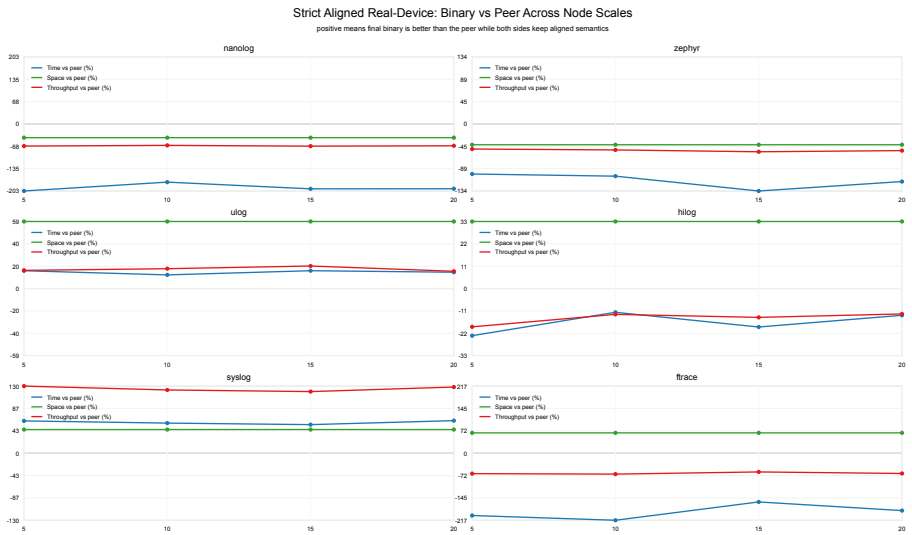

图 **4.12** 严格语义对齐真实多设备场景中 binary 相对 peer 的节点扫描差值

结合表4.6、表4.7以及图4.9、图4.10、图4.11、图4.12可以发现，真实多设备场景下的分组特征比本地多设备更稳定：syslog/ulog 综合为正，nanolog/zephyr 持续处于负区，ftrace 与 hilog 呈现“空间与时间吞吐分化”的结果结构。该现象对应第 3 章中冷启动项与等待项共同作用的多设备模型，也说明真实多设备结果不能由单机或本地多设备结果直接线性外推。

4.4 理论对照与机制讨论

4.4.1 空间交叉点分析

为了回答“日志量增加到什么程度，Optbinlog 的空间性能会优于其他模式”这一问题，本文在严格语义对齐口径下新增 N 扫描实验：单设备单进程，模式固定为 text_semantic_like / binary / peer native mode，记录数扫描点为 1,2,5,10,20,50,100,200,500,1000,5000,20000,100000，每点 repeats=2、warmup=0，统计指标为 total_bytes 均值。交叉点定义为“最早满足 binary_bytes ≤ peer_bytes 的 N”。

表 **4.8** 日志数据量扫描下 binary 相对 peer 的空间交叉点结果

分组	经验交叉点 N^*	线性拟合交叉点	趋势解读
syslog	10	6.0	$N \approx 10$ 附近出现交叉，后续保持空间正收益
nanolog	未出现（至 100000）	−43.9	正 N 区间无交叉，binary 空间持续劣势
zephyr	未出现（至 100000）	−45.2	正 N 区间无交叉，binary 空间持续劣势
ulog	10	9.9	$N \approx 10$ 即出现交叉，后续保持空间正收益
hilog	50	29.7	$N \approx 30 \sim 50$ 出现交叉，小 N 阶段 peer 仍占优

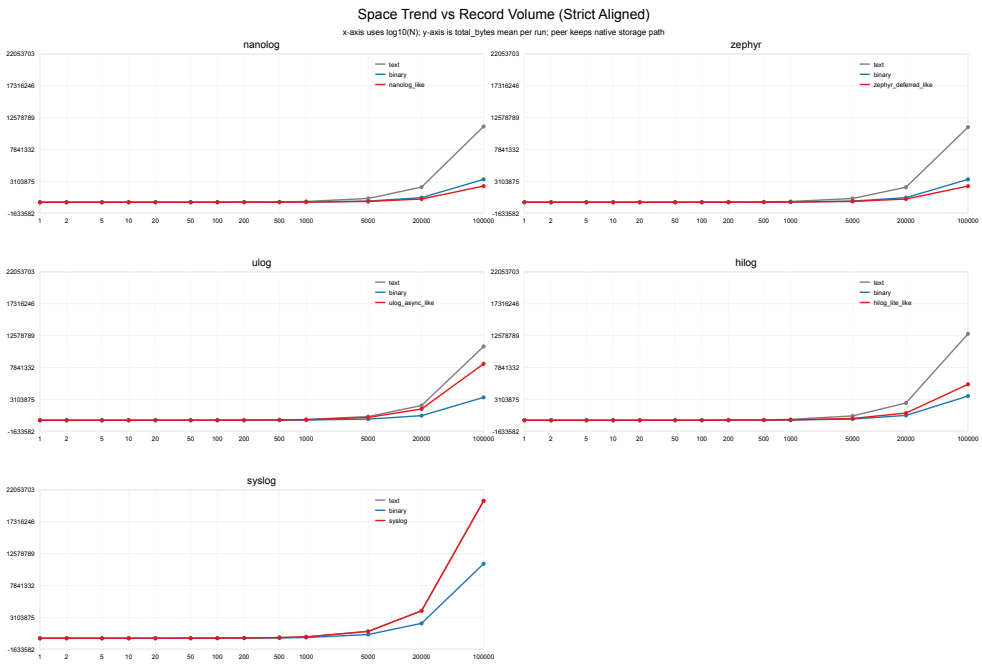

图 **4.13** 日志数据量扫描下各模式空间曲线

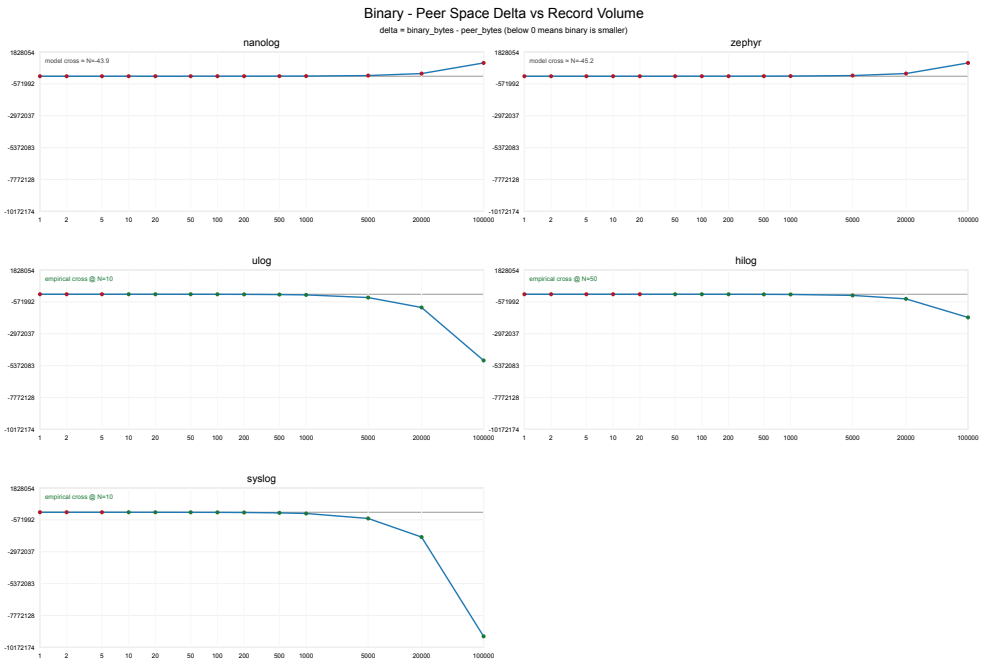

图 **4.14** 日志数据量扫描下 binary 相对 peer 的空间差值

如表4.8以及图4.13、图4.14所示，空间维度呈现出清晰的分组结构。syslog/ulog 在较小日志量（约 $N = 10$）即可完成反超，说明文本展开与系统路径带来的增量字节会较快累积；hilog 在 $N \approx 30 \sim 50$ 左右完成反超，表明轻量头路径在小样本阶段仍保留一定优势；nanolog/zephyr 两组在扫描上限 $N = 100000$ 仍未出现交叉，说明其原生路径的单位记录空间效率在当前语义组下仍然更强。该结果为后续工程部署提供了更直接的阈值判据。

4.4.2 机制解释与边界讨论

为了更直接解释上述三层场景的排序差异，本文继续使用严格公平消融运行作为机制解释层，但不再将其作为主体结论数据集。该数据集给出的关键结果如下：

表 **4.9** 消融实验中的冷路径与校验影响

分组	binary 冷路径 prep(ms)	binary_nocrc 时间改善%
syslog	8.214	+2.45%
nanolog	0.680	-2.82%
zephyr	0.703	+22.60%
ulog	0.713	+4.59%
hilog	0.827	+0.04%

（续表 4.9）

分组	binary 冷路径 prep(ms)	binary_nocrc 时间改善%

表 4.10 消融实验中的字符串编码影响

分组	binary_varstr 时间改善%	binary_nocrc_varstr 时间改善%
syslog	-2.41%	+3.89%
nanolog	-6.86%	+0.38%
zephyr	+4.36%	+16.09%
ulog	-8.85%	-4.41%
hilog	+3.63%	+3.58%

结合表4.9、表4.10以及前文主结果，可以将机制解释概括为以下三点：1. 校验开销并非所有语义组的统一主导项。binary_nocrc 的改善幅度在不同分组之间差异明显，说明关闭校验并不会自动带来一致收益，受益更显著的往往是紧凑定长路径。2. 冷路径差异仍然存在，但在单机阶段已不再只集中于文本型 peer。syslog 的 prep_ms 仍显著高于其他组，而 nanolog/zephyr/ulog/hilog 的 prep_ms 已收敛到较小区间，说明单机差异正更多转向热路径组织与字段编码方式。3. auto-varstr 的收益具有明显的语义组相关性。zephyr/hilog 中变长字符串仍可能带来正收益，而 nanolog/ulog/syslog 中单独开启 varstr 可能转负，说明变长编码并不是所有 schema 的统一最优解，必须结合字段分布加以判断。

从理论对照角度看，本章结果与第 3 章模型总体保持一致：空间口径随 N 近似线性增长，反映出不同语义组的 B_m 差异；本地多设备结果主要体现 $W_m(D,r)$ 的波动，而真实多设备结果则进一步叠加了冷启动项与调度噪声；空间交叉点实验则给出了 $S_{0,m}$ 与 B_m 联合作用下的经验阈值。换言之，单机结果、本地多设备结果与节点级真实多设备结果并非彼此割裂，而是分别对应第 3 章模型中的不同主导项。

本文的主证据链由三部分构成：其一，严格公平层主数据给出 text_semantic_like / binary / peer native mode 在 single、本地多设备与真实多设备三层条件下的比较结果；其二，初始化竞争验证保证共享格式文件在并发场景中的正确性；其三，日志量扫描实验验证空间交叉点是否存在。该证据链使本文结论同时具备指标可复核、机制可解释和阈值可复现三重特征。需要

指出的是，工程近似层在本章中仅用于补充解释，不承担最终排序结论，因此相关外推也必须限定在本文的实验边界之内。

4.5 本章小结

本章在统一实验环境与统计口径下，从单设备过程观测、单机、本地多设备和节点级真实多设备四个层次对 Optbinlog 进行了系统评估，并结合空间交叉点实验与消融结果对主要现象进行了说明。4.1.4 节的运行面板结果为“事件写入—状态迁移—异常恢复”提供了可复核过程证据；主统计结果则表明，在单机场景中，binary 相对文本基线表现出稳定正收益，相对 syslog/ulog/hilog 也已形成较明确优势，而相对 nanolog/zephyr 仍主要受单位记录常数项制约；在多设备场景中，本地多设备结果主要表现为等待项放大带来的时间与吞吐波动，节点级真实多设备结果则进一步稳定地呈现出 syslog/ulog 为优势组、hilog 为过渡组、nanolog/zephyr 与 ftrace 为高压竞争组的分化结构。

空间交叉点实验进一步说明，不同语义组的空间反超阈值并不一致：syslog/ulog 在较小日志量下即可完成反超，hilog 需要更高日志量，而 nanolog/zephyr 在当前扫描范围内仍未出现交叉。结合消融实验可以看出，当前结果差异主要来源于冷路径准备、字符串编码策略、校验开销以及多设备场景下等待项的共同作用。总体而言，本章实验结论与第 3 章模型保持一致，也表明后续优化仍应聚焦冷路径压缩、单位记录常数控制和多设备等待项抑制。

5 结论与展望

5.1 本文主要工作与结论

本文围绕嵌入式场景下“结构化语义、性能开销与并发一致性”这一核心矛盾，完成了 Optbinlog 的设计、建模与验证。回顾全文，本文的主要工作与结论如下：

1. 设计并实现了基于共享格式文件的二进制日志机制。本文构建了由共享 schema、偏移访问、自描述记录帧与记录级校验组成的完整实现路径，并通过初始化状态机约束共享元数据的并发创建与复用语义。
2. 建立了时间、空间与吞吐的一体化分析模型。围绕固定项、边际项和并发等待项，形成了可与实验指标直接映射的理论表达，为单机、多设备与日志量扩展场景提供了统一解释框架。
3. 完成了统一口径下的系统实验验证。基于严格语义对齐数据，本文在单机、本地多设备与真实多设备代理场景下开展对比，并通过初始化竞争验证与空间交叉点扫描形成闭环证据链。
4. 给出了分组化的对比结论与适用边界。实验结果表明，binary 相对 text 在全部语义组中保持稳定正收益；在 peer 对比中，syslog/ulog 表现为稳定优势组，hilog 呈现过渡特征，nanolog/zephyr 在当前节点级口径下仍未完成反超；空间交叉点则呈现明显语义组相关性。

综合来看，本文提出的机制与模型能够较好解释最新实验现象，说明 Optbinlog 更适用于长期在线、强调跨进程一致性、结构化语义与结果可追溯性的嵌入式网关和边缘控制场景。

5.2 未来工作展望

尽管本文已在机制设计与实证分析方面取得阶段性结果，但受平台条件、复现实验范围与观测粒度限制，仍有进一步完善空间。未来工作可从以下几个方向展开：

1. 扩展真实平台验证范围。当前 Linux 结果仍以 VM 节点代理为主，后续可进一步迁移到 Raspberry Pi、i.MX 等真实 ARM 板卡及 bare-metal Linux 环境，以提升结论外部有效性。

2. 推进外部系统全栈可复现对比。在现有代码级阅读与算法级复现基础上，补全更多外部系统官方工程的端到端编译与运行复现，提升横向结论的工程说服力。
3. 补充长生命周期稳态评估。现阶段严格公平套件采用“新进程 + 单次写入”统计口径，后续可引入 steady-state 长时运行基准，进一步量化进程内 shared/tag cache 的持续收益。
4. 深化微观性能机理分析。结合 perf、调度轨迹和 CPU 微架构指标，建立更细粒度的热点归因链路，进一步解释异常点与跨场景波动来源。
5. 完善可靠性与恢复评估体系。围绕损坏、截断、异常注入与恢复路径，构建更系统的鲁棒性测试矩阵，使性能结论与可恢复性结论形成更强一致性。

参考文献

[1] Linux Kernel Documentation. Lockless Ring Buffer Design[EB/OL]. https://www.kernel.org/doc/html/latest/trace/ring-buffer-design.html.

[2] Linux Kernel Documentation. user_events[EB/OL]. https://www.kernel.org/doc/html/latest/trace/user_events.html.

[3] LTTng Project. LTTng Documentation: Concepts[EB/OL]. https://lttng.org/docs/#doc-concepts.

[4] Zephyr Project. Logging[EB/OL]. https://docs.zephyrproject.org/latest/services/logging/index.html.

[5] Zephyr Project. Kconfig.mode[EB/OL]. https://pigweed.googlesource.com/third_party/github/zephyrproject-rtos/zephyr/+/refs/heads/upstream/collab-sdk-dev/subsys/logging/Kconfig.mode.

[6] Zephyr Project. log_msg.c[EB/OL]. https://pigweed.googlesource.com/third_party/github/zephyrproject-rtos/zephyr/+/refs/heads/main/subsys/logging/log_msg.c.

[7] systemd. Journal File Format[EB/OL]. https://systemd.io/JOURNAL_FILE_FORMAT/.

[8] Yang S, Park S J, Ousterhout J. NanoLog: A Nanosecond Scale Logging System[C]//2018 USENIX Annual Technical Conference (USENIX ATC 18). Boston, MA: USENIX Association, 2018:335-350.

[9] PlatformLab. NanoLogCpp17.h[EB/OL]. https://raw.githubusercontent.com/PlatformLab/NanoLog/master/runtime/NanoLogCpp17.h.

[10] PlatformLab. RuntimeLogger.cc[EB/OL]. https://raw.githubusercontent.com/PlatformLab/NanoLog/master/runtime/RuntimeLogger.cc.

[11] Android Open Source Project. event.logtags[EB/OL]. https://android.googlesource.com/platform/system/logging/+/refs/tags/android-13.0.0_r33/logcat/event.logtags.

[12] RT-Thread. ulog 文档 [EB/OL]. https://rt-thread.github.io/rt-thread/page_component_ulog.html.

[13] RT-Thread. ulog.c[EB/OL]. https://raw.githubusercontent.com/RT-Thread/rt-thread/master/components/utilities/ulog/ulog.c.

[14] OpenHarmony. HiLog Lite README[EB/OL]. https://gitee.com/openharmony/hiviewdfx_hilog_lite/blob/master/README_zh.md.

[15] Linux Kernel. ring_buffer.c[EB/OL]. https://raw.githubusercontent.com/torvalds/linux/master/kernel/trace/ring_buffer.c.

[16] systemd. journald-server.c[EB/OL]. https://github.com/systemd/systemd/blob/main/src/journal/journald-server.c.

[17] Sigelman B H, Barroso L A, Burrows M, et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure[R]. Google Technical Report dapper-2010-1, 2010.

[18] Cantrill B M, Shapiro M W, Leventhal A H. Dynamic Instrumentation of Production Systems[C]. USENIX Annual Technical Conference, 2004.

[19] Zhu J, He P, Fu Q, et al. Learning to Log: Helping Developers Make Informed Logging Decisions[C]. ICSE, 2015.

[20] OpenTelemetry. Logs Data Model[EB/OL]. https://opentelemetry.io/docs/specs/otel/logs/data-model/.

[21] Chen T, Song Q, Zhu T, et al. Kellect: A Kernel-based efficient and lossless event log collector for Windows security[J]. Computers & Security, 2025, 150:104203. DOI:10.1016/j.cose.2024.104203.

[22] Balakrishnan R, Sahoo R K. Lossless compression for large scale cluster logs[C]. IPDPS, 2006.

[23] Binlog. Internals[EB/OL]. https://binlog.org/Internals.html.

[24] 闫力, 夏伟. 基于机器学习的日志异常检测综述 [J]. 计算机系统应用, 2022, 31(9):57-69. DOI:10.15888/j.cnki.csa.008661.

[25] 房笑宇, 曹陈涵, 夏彬. 基于注意力机制的大规模系统日志异常检测方法 [J]. 南京大学学报 (自然科学), 2021, 57(5):785-792. DOI:10.13232/j.cnki.jnju.2021.05.008.

[26] 杨瑞朋, 屈丹, 朱少卫, 黄浩. 日志异常检测技术研究 [J]. 信息工程大学学报, 2019(5):610-615. DOI:10.3969/j.issn.1671-0673.2019.05.018.

[27] 姜开达, 章思宇, 孙强. 基于 Hadoop 的校园网站日志系统的设计与实现 [J]. 华东师范大学学报 (自然科学版), 2015(S1):126-131. DOI:10.3969/j.issn.1000-5641.2015.z1.021.

[28] 杜宇峰, 任欣, 王黎. 基于 ARM 嵌入式平台下 Linux 驱动程序开发 [J]. 无线电工程, 2008(6):53-55.

[29] 王鹏, 苗启广, 杨小伟. 基于 ARM 的嵌入式软件系统平台实现 [J]. 无线电工程, 2009(8):49-51.

[30] 赵海峰, 马厚勇, 梁斌. 基于嵌入式 Linux 的 WEB 监控系统设计 [J]. 无线电工程, 2010(4):58-61.

致谢

四年时光易逝，未来无限可期。临别之际，谨以此文聊表感激。

为武汉大学全体教职人员致谢。你们的托举，保证了我们平稳充实的学业旅途。

为我的指导老师武小平教授致谢。您的帮助、指导与建议，贯穿了我整个大学阶段的学习与生活，不胜感激！

为我的家人致谢。感谢父母的支持，支持我每一个天马行空的想法，支持我每一段充满未知的旅程。爱你们！

为我的朋友们致谢。身在北京、上海、洛杉矶的朋友们，感谢每一次相聚，期待下一次相会！

为我的伴侣致谢。感谢相遇、相知与相恋，感谢陪伴、理解与包容，感谢帮助、激励与督促。天高路远，来日方长！

感谢自己，感谢每一次决定、每一次坚持、每一次不悔。

三月微风吹不散四月的雾，六月细雨落不进五月的土。身处当下，再向你们道一声：感谢！

附录

A 关键代码片段

代码清单 A.1 共享元数据初始化与发布点（src/optbinlog_shared.c）

```
/* src/optbinlog_shared.c */
header->state = OPTBINLOG_INITIALIZING;
memcpy(header->magic, OPTBINLOG_SHARED_MAGIC, 8);
header->header_version = OPTBINLOG_SHARED_HEADER_VERSION;
header->num_arrays = (unsigned int)num_arrays;
header->tag_count = (unsigned int)tag_count;
header->schema_hash = schema_hash;

header->state = OPTBINLOG_INITIALIZED;
if (msync(base, total_size, MS_SYNC) != 0) {
    fprintf(stderr, "msync shared failed: %s\n",
            strerror(errno));
    goto fail_cleanup_created;
}
```

代码清单 A.2 记录帧编码与校验写出（src/optbinlog_binlog.c）

```
/* src/optbinlog_binlog.c */
uint32_t frame_header = (uint32_t)payload_size;
if (varstr) {
    frame_header |= OPTBINLOG_FRAME_VARSTR_BIT;
}
frame_header |= ((uint32_t)checksum_type
                 << OPTBINLOG_FRAME_CHECKSUM_SHIFT);
write_le32(dst, frame_header);

size_t off = sizeof(uint32_t);
memcpy(dst + off, &rec->timestamp, sizeof(int64_t));
off += sizeof(int64_t);
memcpy(dst + off, &tag_id_u16, sizeof(uint16_t));
off += sizeof(uint16_t);
memcpy(dst + off, &ele_count_u8, sizeof(uint8_t));
off += sizeof(uint8_t);

if (compute_frame_checksum(checksum_type,
                           dst,
                           dst + sizeof(uint32_t),
                           (size_t)payload_size,
                           &crc) != 0) {
    return -1;
}
write_le32(dst + off, crc);
```

代码清单 A.3 Tag 查找映射（src/optbinlog_shared.c）

```
/* src/optbinlog_shared.c */
OptbinlogEventTag*
optbinlog_lookup_tag(void* base,
                     OptbinlogSharedTag* header,
                     int tag_id, int icnt) {
```

```
    if (tag_id < 0) return NULL;
    int arr = tag_id / OPTBINLOG_EVENT_TAG_ARRAY_LEN;
    int idx = tag_id % OPTBINLOG_EVENT_TAG_ARRAY_LEN;
    if (idx < 0 || idx >= OPTBINLOG_EVENT_TAG_ARRAY_LEN) return NULL;
    if (arr < 0 || arr >= (int)header->num_arrays) return NULL;

    OptbinlogBitmap* bitmap =
        (OptbinlogBitmap*)((uint8_t*)base + header->bitmap_offset);
    if (!optbinlog_bitmap_get(&bitmap[arr], idx)) return NULL;

    int arrayoffset = 0;
    for (int i = 0; i < arr; i++) {
        arrayoffset += bitmap_count_ones(&bitmap[i]);
    }

    int rank = bitmap_rank_inclusive(&bitmap[arr], idx);
    if (rank <= 0) return NULL;
    int slot = arrayoffset + rank - 1;
    if (slot < 0 || slot >= (int)header->tag_count) return NULL;

    OptbinlogEventTag* tags =
        (OptbinlogEventTag*)((uint8_t*)base + header->eventtag_offset);
    OptbinlogEventTag* tag = &tags[slot];
    if (tag->tag_index != (unsigned int)tag_id) return NULL;
    if (icnt != -1 && (int)tag->tag_ele_num != icnt) return NULL;
    return tag;
}
```

代码清单 A.4 断电坏尾修复核心流程（src/optbinlog_binlog.c）

```
/* src/optbinlog_binlog.c */
uint32_t frame_header = read_le32(len_buf);
OptbinlogChecksumType checksum_type =
    (OptbinlogChecksumType)(frame_header >> OPTBINLOG_FRAME_CHECKSUM_SHIFT);
uint32_t payload_len = frame_header & OPTBINLOG_FRAME_LEN_MASK;
if (payload_len < OPTBINLOG_MIN_PAYLOAD_LEN ||
    payload_len > OPTBINLOG_MAX_PAYLOAD_LEN) {
    dirty_tail = 1;
    break;
}
if (!checksum_type_valid(checksum_type)) {
    dirty_tail = 1;
    break;
}

if (read_exact(fp, payload, (size_t)payload_len) != 0) {
    dirty_tail = 1;
    break;
}
if (read_exact(fp, crc_buf, sizeof(crc_buf)) != 0) {
    dirty_tail = 1;
    break;
}

uint32_t expect_crc = read_le32(crc_buf);
uint32_t got_crc = 0u;
if (compute_frame_checksum(checksum_type, len_buf, payload,
                           (size_t)payload_len, &got_crc) != 0) {
    hard_error = 1;
    break;
}
if (expect_crc != got_crc) {
```

```
    dirty_tail = 1;
    break;
}

valid_bytes += sizeof(uint32_t) + (size_t)payload_len + sizeof(uint32_t);
/* 检测到坏尾后，按最后一条完整记录截断。 */
if (dirty_tail && valid_bytes < total) {
    if (truncate(log_path, (off_t)valid_bytes) != 0) {
        return -1;
    }
}
```